ПОШАГОВАЯ РАБОТА ПРОЕКТА

Что именно делает ядро EH/Unwind Analyzer по шагам и зачем нужен каждый шаг

Файл называется “ДЕЙСТВИЯ РЕВЕРСА.pdf” только потому, что так была задана папка документов. По содержанию это не инструкция для человека и не ручной ритуал проверки. Это разбор внутренней работы программы: какие данные она получает, какие проверки выполняет, какие структуры строит и какой результат передает дальше.

О чем этот документ

Документ отвечает на вопрос: что делает проект после запуска. Он идет от командной строки и входного PE-файла к PE-парсингу, .pdata, .xdata, UNWIND_INFO, эвристикам, динамическому слою Windows, Ghidra-разметке и отчетам JSON/HTML/CSV.

Короткая формула

EH/Unwind Analyzer принимает PE x64 файл, строит корректную карту образа, извлекает EH/unwind-метаданные, проверяет их структурные инварианты, превращает нарушения в признаки, объединяет признаки в score/confidence/verdict и сохраняет результат в отчеты.

Главная мысль

Проект является системой анализа, а не голым парсером. Парсер только читает формат. Система анализа дополнительно проверяет смысл прочитанных структур: не выходят ли адреса за границы, не пересекаются ли диапазоны функций, корректно ли разобрана UNWIND_INFO, не лежит ли metadata в странной секции, не доминирует ли один handler и подтверждается ли статика динамическими событиями.

Как читать дальнейшие страницы

Каждый пункт дальше имеет одинаковую структуру: “Вход шага”, “Что делает проект”, “Смысл шага”, “Что проверяется внутри”, “Если проблема”, “Результат шага”. Это сделано специально, чтобы было видно не только действие программы, но и причина, почему действие нужно.

1. Общая карта выполнения

Ниже не действия аналитика, а путь самой программы. Каждый следующий шаг опирается на результат предыдущего. Если ранний шаг ошибся, все дальнейшие адреса, признаки и отчеты стали бы недостоверными, поэтому проект начинает с проверки основы.

Этап	Что получает	Что делает	Что отдает дальше
CLI	командную строку	выбирает режимы анализа и пути отчетов	структуру настроек Args
Входной файл	путь к PE-файлу	читает файл целиком в память	байтовый буфер Buf
PE-слой	Buf	проверяет MZ, PE, COFF, Optional Header, секции	PeImage: карта образа
.pdata	PeImage и Buf	находит Exception Directory и читает RUNTIME_FUNCTION	массив функций rf
.xdata	rf и PeImage	по каждому UnwindInfoAddress разбирает UNWIND_INFO	массив UnwindSummary
Эвристики	PeImage, rf, UnwindSummary	считает нарушения, корреляции, score, confidence	HeuristicReport
Динамика	target exe и статическая карта	на Windows запускает target под debugger и собирает runtime-события	DynamicReport
Отчеты	все результаты	пишет CSV, Ghidra CSV, JSON, HTML и консольный вывод	готовые артефакты

2. Подробный маршрут работы проекта

Дальше идет основной раздел. Его надо читать как трассировку выполнения: программа движется сверху вниз. В каждом пункте описано не то, что должен сделать человек, а то, что делает код проекта.

Шаг проекта 1. Программа принимает командную строку и определяет режим работы

Вход шага: массив аргументов argc/argv: путь к файлу и опции вроде --dump, --heuristics, --json, --html, --dynamic, --ghidra-csv

Что делает проект: main.c вызывает parse_args, заполняет структуру Args и задает значения по умолчанию: follow_chain=true, max_chain_depth=8, timeout_ms=15000, trace_steps=64.

Смысл шага: До чтения бинарника проект должен понять, какую работу от него попросили: только вывести краткую информацию, дополнительно разобрать операции, посчитать эвристики, запустить динамическую проверку или сформировать отчеты.

Что проверяется внутри: Проверяется корректность числовых параметров: --limit не отрицательный, --max-chain-depth в диапазоне 1..16, --timeout-ms больше нуля, --trace-steps в диапазоне 0..256. Также отделяются аргументы самого анализатора от аргументов target-программы после разделителя --.

Если проблема: при неизвестной опции или неправильном числе программа показывает usage и завершает работу с кодом ошибки 2. Это защита от молчаливого запуска не того режима.

Результат шага: структура Args, которая управляет всеми последующими ветками анализа.

Шаг проекта 2. Программа читает анализируемый файл в байтовый буфер

Вход шага: путь к PE-файлу из первого позиционного аргумента

Что делает проект: util-слой открывает файл, считывает его целиком и формирует Buf: указатель на байты и размер файла.

Смысл шага: Все дальнейшие проверки должны работать с одним стабильным снимком файла. Проект не должен перечитывать файл кусками и зависеть от того, изменился ли файл на диске во время анализа.

Что проверяется внутри: Проверяется сам факт чтения файла. Если файл недоступен, путь неверный или чтение сорвалось, дальнейший PE-анализ невозможен.

Если проблема: программа печатает Failed to read file и завершает работу с кодом ошибки 1.

Результат шага: Buf становится исходным материалом для PE-парсера.

Шаг проекта 3. Проект проверяет DOS-заголовок и смещение до PE-заголовка

Вход шага: байтовый буфер Buf

Что делает проект: pe_parse читает первые два байта и ждет сигнатуру MZ, затем читает поле e_lfanew по смещению 0x3c.

Смысл шага: PE-файл начинается с DOS-заголовка, а e_lfanew указывает, где лежит настоящий NT/PE-заголовок. Без этого нельзя безопасно найти COFF Header и Optional Header.

Что проверяется внутри: Проверяется MZ-сигнатура и то, что e_lfanew не уводит за конец файла.

Если проблема: если MZ неверный или e_lfanew выходит за границы, pe_parse возвращает ошибку Not a valid MZ/PE file или PE header offset out of range.

Результат шага: валидное смещение lfanew, от которого можно читать NT headers.

Шаг проекта 4. Проект проверяет PE-сигнатуру и COFF Header

Вход шага: смещение lfanew и байтовый буфер

Что делает проект: pe_parse читает PE signature, затем Machine, NumberOfSections и SizeOfOptionalHeader из COFF Header.

Смысл шага: Этот шаг отвечает на вопрос: перед нами действительно PE-образ нужной архитектуры или случайный/неподдерживаемый файл. Проект заточен под PE x64, потому что EH/unwind-метаданные x64 устроены через .pdata/.xdata.

Что проверяется внутри: Проверяется сигнатура PE, читаемость COFF Header и значение Machine. Поддерживается IMAGE_FILE_MACHINE_AMD64; 32-битные PE и другие архитектуры отвергаются.

Если проблема: если Machine не AMD64, программа завершает PE parse error с объяснением Unsupported Machine.

Результат шага: подтверждение, что дальше можно применять именно x64 unwind-правила.

Шаг проекта 5. Проект проверяет Optional Header как PE32+ и читает базовые поля образа

Вход шага: COFF Header и позиция Optional Header

Что делает проект: pe_parse проверяет magic 0x20b, затем читает ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders и NumberOfRvaAndSizes.

Смысл шага: Эти поля задают координатную систему всего анализа. ImageBase нужен для связи RVA и VA, SizeOfImage ограничивает допустимые RVA, alignment-поля нужны для проверки секций, NumberOfRvaAndSizes говорит, есть ли запись Exception Directory.

Что проверяется внутри: Проверяется размер Optional Header, PE32+ magic, ненулевой SizeOfImage, корректный SizeOfHeaders, ненулевые выравнивания и наличие нужной DataDirectory-записи.

Если проблема: при нарушении проект останавливает PE-парсинг, потому что без этих полей любой вывод о .pdata/.xdata станет недостоверным.

Результат шага: PeImage получает базовые параметры загруженного образа.

Шаг проекта 6. Проект находит запись Exception Directory в Data Directory

Вход шага: Optional Header и NumberOfRvaAndSizes

Что делает проект: pe_parse вычисляет смещение записи IMAGE_DIRECTORY_ENTRY_EXCEPTION и читает два поля: exception_rva и exception_size.

Смысл шага: Exception Directory - это официальный указатель PE на таблицу RUNTIME_FUNCTION. Для x64 именно она связывает диапазоны кода с unwind-описаниями.

Что проверяется внутри: Проверяется, что запись DataDirectory физически помещается в Optional Header. Затем проверяется, что пара RVA/Size либо корректно отсутствует, либо не выходит за SizeOfImage.

Если проблема: если запись лежит вне Optional Header, PE-парсинг падает; если диапазон Exception Directory подозрительный, это запоминается как structural issue в PeImage.

Результат шага: в PeImage сохраняются exception_rva, exception_size и флаг exception_directory_oob.

Шаг проекта 7. Проект читает таблицу секций и строит карту RVA -> file offset

Вход шага: Section headers после Optional Header

Что делает проект: pe_parse читает для каждой секции имя, VirtualSize, VirtualAddress, SizeOfRawData, PointerToRawData и Characteristics.

Смысл шага: Секции связывают виртуальные адреса образа с байтами файла. Почти все дальнейшие действия требуют ответа: данный RVA можно прочитать из файла или он указывает в виртуальную область, которой в файле нет.

Что проверяется внутри: Проверяются границы raw-диапазонов, virtual-диапазонов, пересечения секций, монотонность VirtualAddress, выравнивание raw/virtual полей, согласованность SizeOfHeaders, SizeOfImage и ImageBase.

Если проблема: нарушения не всегда останавливают весь анализ, но сохраняются как счетчики section_raw_oob, section_virtual_oob, section_raw_overlaps, section_virtual_overlaps, section_misaligned. Потом эти счетчики попадут в эвристики.

Результат шага: PeImage получает массив PeSection и функцию pe_rva_to_fileoff начинает работать осмысленно.

Шаг проекта 8. Проект переводит RVA в file offset только через проверенную карту секций

Вход шага: любой RVA, который нужно прочитать: exception_rva, unwind_rva, handler_rva для контекста

Что делает проект: pe_rva_to_fileoff ищет секцию, чей virtual range покрывает RVA. Если секция найдена, вычисляет delta=RVA-VirtualAddress и fileoff=PointerToRawData+delta. Если RVA попадает в headers, он мапится напрямую.

Смысл шага: Это предотвращает типичную ошибку: обращаться с RVA как с file offset. В PE это разные координаты. RVA живет в загруженном образе, file offset - позиция байта в файле.

Что проверяется внутри: Проверяется, что RVA попал в секцию или заголовки, delta не указывает за SizeOfRawData, сложение не переполняет uint32, а итоговый file offset не выходит за размер файла.

Если проблема: если RVA не мапится, соответствующая структура считается нечитаемой. Для Exception Directory это structural error, для UNWIND_INFO - parse failure.

Результат шага: безопасный file offset или отказ от чтения.

Шаг проекта 9. Проект читает Exception Directory как массив RUNTIME_FUNCTION

Вход шага: exception_rva, exception_size, PeImage и Buf

Что делает проект: unwind_read_runtime_functions_ex сначала обрабатывает случай отсутствия директории: RVA=0 и Size=0 означает пустой массив. Затем проверяет, что RVA и Size согласованы, директория мапится в file offset, помещается в файл, имеет размер не меньше 12 и кратна 12.

Смысл шага: Одна запись RUNTIME_FUNCTION занимает 12 байт: BeginAddress, EndAddress, UnwindInfoAddress. Поэтому размер директории должен делиться на 12 без остатка.

Что проверяется внутри: Проверяется malformed-пара RVA/Size, читаемость диапазона, размер, кратность 12 и успешное выделение памяти под массив.

Если проблема: при структурной проблеме main не делает вид, что все хорошо: помечает exception_directory_oob, печатает ошибку и продолжает с частичным structural report, если это возможно.

Результат шага: массив RuntimeFunction rf и количество записей rf_count.

Шаг проекта 10. Проект разбирает каждую запись RUNTIME_FUNCTION

Вход шага: 12-байтовые элементы Exception Directory

Что делает проект: для каждой записи проект читает BeginAddress, EndAddress и UnwindInfoAddress как 32-битные RVA и кладет их в RuntimeFunction.

Смысл шага: Эти три поля образуют связь: диапазон кода [BeginAddress, EndAddress) обслуживается unwind-описанием по UnwindInfoAddress.

Что проверяется внутри: На этапе чтения проект не пытается заранее исправлять записи. Он сохраняет то, что лежит в файле, а валидность диапазонов проверяет позже в эвристиках.

Если проблема: если чтение конкретной записи выходит за границы файла, весь массив считается malformed, потому что таблица физически повреждена.

Результат шага: сырой, но структурно прочитанный массив function table.

Шаг проекта 11. Проект запускает разбор UNWIND_INFO для каждой функции

Вход шага: rf[i].unwind_rva для каждой RUNTIME_FUNCTION

Что делает проект: main.c проходит циклом по rf_count и вызывает unwind_parse. Для каждой функции создается UnwindSummary и отдельный флаг us_ok.

Смысл шага: Проекту недостаточно знать, что функция имеет UnwindInfoAddress. Нужно проверить, что по этому адресу действительно лежит валидная структура UNWIND_INFO, которую можно использовать для восстановления стека.

Что проверяется внутри: Для каждой записи отдельно фиксируется успех или ошибка. Это важно: один поврежденный UNWIND_INFO не должен уничтожать анализ всех остальных функций.

Если проблема: если конкретный UNWIND_INFO не разбирается, us_ok[i]=false, а текст причины сохраняется в parse_error. Позже это станет признаком unwind_parse_fail.

Результат шага: массив UnwindSummary и массив bool us_ok той же длины, что rf.

Шаг проекта 12. Проект читает заголовок UNWIND_INFO

Вход шага: UnwindInfoAddress конкретной функции

Что делает проект: unwind_parse переводит unwind_rva в file offset, читает первые 4 байта и извлекает Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister и FrameOffset.

Смысл шага: Заголовок говорит, как читать хвост структуры: сколько unwind-кодов есть, есть ли handler, есть ли chained info, используется ли frame pointer.

Что проверяется внутри: Проверяется выравнивание unwind_rva, маппинг в файл, наличие 4 байтов, версия 1 или 2, допустимые флаги, несовместимые комбинации flags, корректность frame register/frame offset.

Если проблема: если заголовок не читается или версия неизвестна, структура помечается как invalid, аномалии сохраняются, парсинг этой записи прекращается.

Результат шага: первичная UnwindSummary с полями version, flags, prolog_size, count_codes, frame_reg, frame_off.

Шаг проекта 13. Проект вычисляет размер массива unwind-кодов и проверяет, что он помещается в файл

Вход шага: CountOfCodes из заголовка UNWIND_INFO

Что делает проект: код считает количество 2-байтовых слотов с учетом выравнивания до четного числа: slots_even=(count+1)&~1. Затем проверяет, что массив слотов физически доступен в файле.

Смысл шага: UNWIND_CODE хранится не как обычный список одинаковых команд: часть операций занимает дополнительные слоты. Но сам хвост структуры после кодов начинается только после четного количества слотов.

Что проверяется внутри: Проверяется, что codes_off + slots_even*2 не выходит за размер файла.

Если проблема: если массив слотов обрезан, выставляется UA_TRUNCATED и parse failure.

Результат шага: безопасный диапазон байтов, по которому можно идти парсером операций.

Шаг проекта 14. Проект идет по каждому unwind opcode и определяет, сколько слотов он занимает

Вход шага: массив UNWIND_CODE

Что делает проект: для каждого стартового слота проект выделяет CodeOffset, UnwindOp и OpInfo. Функция slots_for_code определяет длину операции: например PUSH_NONVOL занимает 1 слот, ALLOC_LARGE - 2 или 3, SAVE_NONVOL_FAR - 3, PUSH_MACHFRAME - 1.

Смысл шага: Без знания длины операции нельзя понять, где начинается следующая операция и где заканчивается список. Это защита от рассинхронизации парсера.

Что проверяется внутри: Проверяется известность opcode, допустимость opinfo для некоторых операций, выход операции за CountOfCodes, порядок CodeOffset, попадание CodeOffset в SizeOfProlog.

Если проблема: если opcode неизвестен, проект помечает UA_BAD_OPCODE и пытается безопасно продолжить как 1 слот. Если операция физически вылезает за CountOfCodes, это уже parse failure.

Результат шага: последовательность разобранных операций или точная причина, почему последовательность повреждена.

Шаг проекта 15. Проект вычисляет смысл unwind-операций для стека и регистров

Вход шага: каждая распознанная unwind-операция

Что делает проект: unwind_parse не просто перечисляет opcodes. Он накапливает fixed_stack_bytes, pushed_nonvol_count, saved_nonvol_mask и saved_xmm_mask.

Смысл шага: Эти поля объясняют, что unwind metadata утверждает о прологе функции: сколько байт стека выделено, какие nonvolatile-регистры сохранены, используются ли XMM-регистры и frame pointer.

Что проверяется внутри: Проверяются некорректные регистры, подозрительные размеры ALLOC_LARGE, странные FAR offsets, PUSH_MACHFRAME opinfo, дубли SET_FPREG, нарушения ABI при --strict-abi.

Если проблема: при нарушениях выставляются битовые флаги аномалий: UA_BAD_OPERAND, UA_OPERAND_SANITY, UA_ABI_REGISTER_CLASS, UA_ABI_SAVE_OFFSET и другие. Часть нарушений не ломает парсинг, но снижает доверие к структуре.

Результат шага: UnwindSummary получает численное описание stack/register effects и маску anomalies.

Шаг проекта 16. Проект читает хвост UNWIND_INFO: handler или chained info

Вход шага: позиция after_codes сразу после выровненного массива unwind-кодов

Что делает проект: если Flags содержит CHAININFO, проект читает встроенную структуру RUNTIME_FUNCTION. Если Flags содержит EHANDLER или UHANDLER, проект читает HandlerRVA.

Смысл шага: Хвост определяет, связана ли функция с обработчиком исключения или переиспользует unwind-описание другой функции через chained unwind.

Что проверяется внутри: Для chain проверяется положительный диапазон, попадание Begin/End/UnwindRVA в SizeOfImage, ненулевой UnwindRVA. Для handler проверяется, что HandlerRVA не выходит за SizeOfImage. Также проверяется недопустимое сочетание CHAININFO с EHANDLER/UHANDLER.

Если проблема: если хвост обрезан, это parse failure. Если хвост читается, но поля странные, выставляются UA_CHAIN_INVALID или UA_HANDLER_OOB.

Результат шага: UnwindSummary получает has_handler, handler_rva, has_chain и chain.

Шаг проекта 17. Проект проходит chained unwind-цепочки с ограничением глубины

Вход шага: UnwindSummary, где установлен флаг CHAININFO

Что делает проект: если follow_chain включен, unwind_parse переходит к chain.unwind_rva, разбирает следующую UNWIND_INFO, объединяет stack/register effects и следит за уже виденными unwind_rva.

Смысл шага: Chained unwind нужен, когда функция ссылается на первичную unwind-информацию другой функции. Но такая цепочка может быть повреждена или намеренно зациклена.

Что проверяется внутри: Проверяется цикл, превышение max_chain_depth, ошибка разбора следующего элемента, несогласованность frame register/frame offset между связанными записями.

Если проблема: при цикле ставится UA_CHAIN_LOOP, при превышении - UA_CHAIN_DEPTH_EXCEEDED, при ошибке вложенного разбора - UA_TRUNCATED или соответствующая аномалия.

Результат шага: итоговый UnwindSummary учитывает не только первую структуру, но и цепочку, если ее можно безопасно пройти.

Шаг проекта 18. Проект формирует статические признаки по PE-структуре

Вход шага: PeImage со счетчиками проблем секций и Exception Directory

Что делает проект: heuristics_analyze переносит structural issues из PE-слоя в HeuristicReport: raw/virtual out-of-bounds, overlaps, misalignment, malformed Exception Directory.

Смысл шага: Если сама PE-карта повреждена, нельзя полностью доверять последующим адресам. Поэтому проект учитывает проблемы секций как отдельную семью evidence: table_integrity.

Что проверяется внутри: Проверяется, есть ли section_raw_oob, section_virtual_oob, raw/virtual overlaps, misalignment, exception_directory_oob, пустая .pdata при наличии executable-секции.

Если проблема: каждое найденное нарушение превращается в finding с code, severity, family, message, representative_rva и affected_count.

Результат шага: HeuristicReport получает первые finding-и и начальный вклад в score.

Шаг проекта 19. Проект проверяет целостность RUNTIME_FUNCTION-таблицы

Вход шага: массив rf и SizeOfImage

Что делает проект: heuristics_analyze проходит все записи и считает invalid ranges, выход Begin/End за SizeOfImage, нулевой или out-of-bounds UnwindInfoAddress, нарушение исходной сортировки.

Смысл шага: Таблица .pdata должна давать однозначную карту от адреса инструкции к unwind metadata. Если диапазоны неправильные, пересекаются или не отсортированы, поиск unwind-записи становится ненадежным.

Что проверяется внутри: Проверяется EndAddress > BeginAddress, Begin/End внутри SizeOfImage, UnwindInfoAddress внутри image и не равен нулю, порядок BeginAddress по исходной таблице.

Если проблема: нарушения становятся признаками invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted.

Результат шага: HeuristicReport получает счетчики table_integrity по function table.

Шаг проекта 20. Проект сортирует копию диапазонов и ищет пересечения и дубликаты

Вход шага: массив rf

Что делает проект: эвристический слой создает временный массив Begin/End, сортирует его по BeginAddress и проверяет duplicate BeginAddress и overlap между соседними диапазонами.

Смысл шага: Даже если исходная таблица была неотсортирована, после сортировки можно увидеть фактическую геометрию диапазонов: пересекаются ли функции и повторяются ли начала.

Что проверяется внутри: Проверяется, равен ли BeginAddress у соседних записей, и меньше ли BeginAddress текущей записи, чем максимальный EndAddress предыдущих.

Если проблема: если найдено пересечение, проект выдает rf_overlaps; если повтор начала - duplicate_begin.

Результат шага: HeuristicReport получает более строгую проверку однозначности диапазонов.

Шаг проекта 21. Проект проверяет целостность разобранных UNWIND_INFO

Вход шага: массив UnwindSummary и флаги us_ok

Что делает проект: heuristics_analyze считает, сколько UNWIND_INFO вообще не разобралось, и сколько разобралось, но содержит аномалии вне чистых ABI-warning.

Смысл шага: Это отдельный слой: .pdata может выглядеть формально читаемой, но .xdata по указанным адресам может быть обрезана, иметь странные opcodes, неверные operands, плохие handlers или сломанные chain-ссылки.

Что проверяется внутри: Проверяется us_ok=false как unwind_parse_fail и маска anomalies без ABI-only битов как unwind_anomalies. ABI-предупреждения учитываются отдельно и не обязательно означают вмешательство.

Если проблема: ошибки превращаются в finding-и unwind_parse_fail и unwind_anomalies с семейством unwind_integrity.

Результат шага: HeuristicReport получает evidence о внутренней согласованности .xdata.

Шаг проекта 22. Проект проверяет размещение unwind metadata и handler targets по секциям

Вход шага: UnwindInfoAddress, HandlerRVA и карта секций PeImage

Что делает проект: для каждого успешно разобранного UNWIND_INFO проект смотрит, в какой секции лежит сам UnwindInfoAddress и куда указывает HandlerRVA, если handler есть.

Смысл шага: Обычная unwind metadata находится в data/metadata-секциях вроде .xdata, .rdata или .pdata. Handler обычно должен указывать на исполняемый код. Нетипичные права секций усиливают подозрение.

Что проверяется внутри: Проверяется: UNWIND_INFO в executable-секции, writable-секции, необычной секции; handler в non-executable/unknown секции; handler в writable-секции.

Если проблема: нарушения становятся finding-и unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section.

Результат шага: HeuristicReport получает evidence family metadata_placement.

Шаг проекта 23. Проект считает повторное использование UnwindInfoAddress и доминирование handler

Вход шага: все rf[i].unwind_rva и все handler_rva из UnwindSummary

Что делает проект: heuristics_analyze использует внутренние hash map счетчики: считает, сколько уникальных UnwindInfoAddress есть и какое максимальное число функций ссылается на один адрес. Отдельно считает, доминирует ли один handler среди многих handler-записей.

Смысл шага: Массовое reuse одной unwind-структуры или одного handler может быть нормальным только в некоторых runtime/compiler-сценариях. В остальных случаях это похоже на синтетическую разметку, dispatcher-style обработку или вмешательство.

Что проверяется внутри: Проверяются ratios: unwind_reuse_ratio, unwind_unique_ratio, handler_reuse_ratio. Сильный reuse требует большого объема n>200, handler dominance требует handler_count>50 и ratio>0.80, плюс контекстные ослабления.

Если проблема: если reuse слабый или изолированный, проект не делает чрезмерный вывод. Он либо не добавляет finding, либо добавляет mitigation о необходимости ручной проверки.

Результат шага: HeuristicReport получает evidence families unwind_reuse и handler_dominance, если сигналы достаточно сильны.

Шаг проекта 24. Проект усиливает вывод только когда признаки подтверждают друг друга

Вход шага: уже найденные finding-и разных семейств

Что делает проект: heuristics_analyze добавляет combo-признаки: подозрительное размещение metadata вместе с handler-проблемами, placement вместе с integrity-проблемами, reuse вместе с dominant handler, writable metadata вместе с writable handler.

Смысл шага: Один слабый сигнал может быть особенностью компилятора. Несколько независимых сигналов, которые указывают в одну сторону, дают более сильное основание для подозрения.

Что проверяется внутри: Проверяются комбинации: unwind_exec_section + handler_nonexec_section, placement + parse/OOB/integrity, reuse + handler_dominance, unwind_writable_section + handler_writable_section.

Если проблема: если комбинация найдена, score увеличивается сильнее, чем от одиночного признака.

Результат шага: HeuristicReport получает корреляционные finding-и, которые объясняют, почему итоговый вывод стал сильнее.

Шаг проекта 25. Проект считает итоговый score, confidence, evidence_level и verdict

Вход шага: все найденные признаки, severity и evidence families

Что делает проект: finalize_verdict переводит сумму весов в suspicion=score/100, считает количество семейств доказательств, strong/medium/weak findings, confidence и requires_manual_review.

Смысл шага: score отвечает на вопрос “насколько много и насколько серьезные признаки найдены”. confidence отвечает на другой вопрос: “насколько уверенно можно интерпретировать эти признаки”. Это разные величины.

Что проверяется внутри: Применяются ослабления: single_signal_only, small_sample_size, placement_only_signal, single_family_only. Verdict выбирается по score: high_suspicion от 70, moderate_suspicion от 40, low_suspicion от 15, иначе no_strong_static_anomalies.

Если проблема: если score=0, проект явно добавляет note: No strong anomalies detected by heuristics. Если признаки слабые или одиночные, проект снижает confidence и требует manual review.

Результат шага: HeuristicReport становится готовым итогом статического слоя.

Шаг проекта 26. Проект при необходимости запускает динамический слой Windows

Вход шага: опция --dynamic, путь к target exe, аргументы после --, статическая карта PeImage/rf/us

Что делает проект: main.c вызывает dynamic_collect_windows. На Windows этот слой запускает target-процесс под Debug API и собирает события, связанные с исключениями, обработчиками и runtime function tables.

Смысл шага: Статика показывает, что записано в файле. Динамика показывает, что происходит при исполнении: регистрируются ли VEH/VCH/UEF, добавляются ли runtime function tables, достигаются ли handler-адреса.

Что проверяется внутри: Проверяются timeout, возможность создать процесс под debugger, возможность обновлять список модулей, ставить breakpoints, наблюдать exceptions, API calls и handler hits.

Если проблема: если dynamic на этой ОС недоступна или запуск не удался, проект пишет Dynamic Failed и выставляет ненулевой exit_code, но статические результаты остаются полезными.

Результат шага: DynamicReport с counters, flags instrumentation_complete/partial и текстовыми details.

Шаг проекта 27. Динамический слой строит карту загруженных модулей target-процесса

Вход шага: pid процесса, запущенного под debugger

Что делает проект: dynamic_win.c через Toolhelp snapshot собирает список модулей: имя, base, size. Карту периодически обновляет и отмечает failures/truncations.

Смысл шага: Чтобы поставить breakpoint на нужную API-функцию или понять, где находится адрес события, проект должен знать базовые адреса DLL и самого target image в runtime-памяти.

Что проверяется внутри: Проверяется успешность snapshot, число модулей, факт усечения внутреннего массива, максимальное observed число модулей.

Если проблема: если список модулей не обновился, instrumentation_complete становится false, а в DynamicReport растет module_refresh_failures.

Результат шага: актуальная ModList для адресной привязки runtime-событий.

Шаг проекта 28. Динамический слой ставит программные breakpoints на интересующие точки

Вход шага: runtime-адреса API и handler targets

Что делает проект: dynamic_win.c читает первый байт по адресу, сохраняет его, пишет 0xCC, сбрасывает instruction cache и логирует self-check: куда breakpoint поставлен и вооружен ли он.

Смысл шага: Breakpoint позволяет проекту не гадать по файлу, а увидеть факт исполнения: target действительно дошел до регистрации VEH/VCH/UEF, добавления function table или handler target.

Что проверяется внутри: Проверяется ReadProcessMemory, WriteProcessMemory, при необходимости VirtualProtectEx, вместимость массива breakpoints и байт self-check.

Если проблема: если breakpoint нельзя поставить, DynamicReport увеличивает breakpoint_set_failures, а instrumentation_complete может стать false.

Результат шага: набор armed breakpoints и доказательная запись в dynamic details.

Шаг проекта 29. Динамический слой обрабатывает debug events и считает runtime-сигналы

Вход шага: события Windows Debug API от target-процесса

Что делает проект: проект обрабатывает исключения, срабатывания breakpoints, module events, завершение процесса и timeout. При попадании в интересную точку увеличивает соответствующие счетчики: exceptions_seen, unwind_handler_hits, veh_hits, dyn_ftable_calls и другие.

Смысл шага: Цель динамики - не заменить статический парсер, а дать второй слой evidence: что из подозрительного или нормального реально проявилось во время исполнения.

Что проверяется внутри: Проверяется, где произошел breakpoint, к какому kind он относится, надо ли восстановить оригинальный байт, сделать single-step и повторно вооружить breakpoint.

Если проблема: если target завершился, завис или не дошел до событий, отчет фиксирует это как наблюдение. Отсутствие события не доказывает отсутствие механизма, если путь исполнения не был покрыт.

Результат шага: DynamicReport с наблюдаемыми runtime-фактами и ограничениями сбора.

Шаг проекта 30. Проект соединяет статический и динамический контекст в отчетах

Вход шага: HeuristicReport и DynamicReport

Что делает проект: report.c при записи JSON/HTML добавляет dynamic assessment и correlation: что найдено статикой, что подтверждено или не подтверждено динамикой, полная ли инструментализация.

Смысл шага: Это нужно, чтобы отчет не выглядел как набор несвязанных чисел. Пользователь должен видеть: статические признаки есть, динамические сигналы есть/нет, сбор полный/частичный, вывод требует проверки или достаточно сильный.

Что проверяется внутри: Проверяется наличие hr_ptr и dr_ptr, metadata, counters, failures и details. В отчете сохраняются как итоговые поля, так и первичные данные.

Если проблема: если динамика не запускалась, JSON/HTML все равно пишутся со статической оценкой. Если динамика запускалась частично, это отражается как ограничение evidence.

Результат шага: единая картина анализа для защиты и дальнейшей интеграции с backend/database.

Шаг проекта 31. Проект формирует CSV для табличного анализа

Вход шага: PeImage, rf, us, us_ok

Что делает проект: report_write_csv пишет строки по функциям: begin/end/unwind, stack/register summary, flags, anomalies, handler/chain и другую информацию.

Смысл шага: CSV нужен для быстрой проверки, фильтрации, сравнения результатов и загрузки в другие инструменты.

Что проверяется внутри: Проверяется корректность аргументов, создание буфера, экранирование CSV-ячеек и успешная атомарная запись файла.

Если проблема: при ошибке записи программа печатает CSV write failed и выставляет exit_code=1.

Результат шага: готовый analysis CSV.

Шаг проекта 32. Проект формирует специальный CSV для Ghidra

Вход шага: те же rf/us/us_ok, но включен ghidra_mode

Что делает проект: report_write_csv в Ghidra-режиме пишет колонки begin_rva,end_rva,name,comment. Name строится как sub_XXXXXXXX, а comment содержит UNWIND-summary и anomalies.

Смысл шага: Ghidra-CSV нужен не как главный отчет, а как мост: перенести найденные диапазоны функций и warning-комментарии прямо в дизассемблер.

Что проверяется внутри: Проверяется, можно ли построить комментарий, есть ли parse failure, есть ли anomalies. При parse failure комментарий явно содержит UNWIND parse failed.

Если проблема: если файл не пишется, программа сообщает Ghidra CSV write failed.

Результат шага: CSV, который понимает ghidra/import_unwind_csv.py.

Шаг проекта 33. Ghidra-скрипт импортирует разметку проекта в текущую программу Ghidra

Вход шага: CSV из --ghidra-csv и открытый Ghidra project

Что делает проект: import_unwind_csv.py проверяет обязательные колонки, берет image base текущей программы, переводит begin/end RVA в адреса Ghidra, создает или обновляет функции, добавляет комментарии и warning bookmarks.

Смысл шага: Сам анализ делает не Ghidra-скрипт, а ядро проекта. Скрипт нужен, чтобы результат анализа стал видимым в reverse engineering UI: адреса функций, комментарии к ним, предупреждения по anomalies.

Что проверяется внутри: Проверяется валидность half-open range, попадание адресов в память Ghidra, отсутствие конфликтов с существующими функциями, сохранение пользовательских имен и комментариев.

Если проблема: конфликтные строки пропускаются и печатаются как skip, чтобы импорт не разрушал уже существующую разметку.

Результат шага: размеченный Ghidra project с функциями, комментариями и bookmarks.

Шаг проекта 34. Проект формирует JSON-отчет как главный машинный результат

Вход шага: все структуры анализа и ReportMetadata

Что делает проект: main.c перед JSON готовит metadata: version, schema version, UTC timestamp, SHA-256 входного файла, размер input, command line, compiler/platform/build mode, strict_abi, follow_chain, max_chain_depth. Затем report_write_json_ex пишет структурированный отчет.

Смысл шага: JSON нужен для backend, базы данных, сравнения анализов, фронтенда и автоматизации. Это главный контракт ядра с внешней системой.

Что проверяется внутри: Проверяется подготовка metadata, расчет SHA-256, сбор command line без раскрытия target-аргументов после --, успешная запись JSON.

Если проблема: если metadata не подготовилась или файл не записался, программа сообщает ошибку и возвращает ненулевой exit_code.

Результат шага: полный JSON с PE-инфой, функциями, unwind summaries, heuristic assessment, dynamic assessment и metadata.

Шаг проекта 35. Проект формирует HTML-отчет для просмотра человеком

Вход шага: те же данные, что для JSON

Что делает проект: report_write_html создает самодостаточный HTML-отчет: обзор, таблицы, признаки, объяснение observed/expected/impact, динамические детали и шаги ручной валидации при необходимости.

Смысл шага: HTML нужен, чтобы не читать сырой JSON глазами. Это форма презентации результата, но источником истины остается структурированный анализ.

Что проверяется внутри: Проверяется экранирование HTML, наличие metadata, список findings, таблицы функций и связанный контекст.

Если проблема: если HTML не записался, программа печатает HTML write failed и выставляет exit_code=1.

Результат шага: готовый HTML-отчет для заказчика, защиты или быстрой проверки.

Шаг проекта 36. Проект печатает краткий консольный итог и dump при запросе

Вход шага: PeImage, rf_count, us/us_ok и CLI-флаги

Что делает проект: report_print_brief всегда печатает файл, Machine, ImageBase, SizeOfImage, SizeOfHeaders, число секций, Exception Directory и количество RUNTIME_FUNCTION. При --dump main печатает строки функций; при --ops дополнительно печатает список unwind operations.

Смысл шага: Консольный вывод нужен для быстрого запуска без файлов отчетов. Это не главный формат интеграции, но удобный способ понять, что анализатор вообще прочитал файл.

Что проверяется внутри: Проверяется limit, чтобы не вывести больше строк, чем есть. При --ops операции пересобираются для читаемого вывода.

Если проблема: если отдельная UNWIND_INFO не разобралась, в dump видно UNWIND: parse failed с причиной.

Результат шага: оперативный терминальный отчет.

Шаг проекта 37. Проект освобождает ресурсы и возвращает итоговый exit code

Вход шага: все выделенные структуры: rf, us, hr, dr, PeImage, StrBuf, input buffer

Что делает проект: main.c освобождает все динамические буферы, секции, отчеты, строки metadata и возвращает exit_code.

Смысл шага: Это важная часть готового инструмента: после анализа не должны оставаться утечки памяти, а вызывающая система должна получить машинный признак успеха или ошибки.

Что проверяется внутри: Проверяется, были ли включены heuristics/dynamic и были ли их структуры инициализированы, чтобы освобождать только реально созданное.

Если проблема: если по ходу анализа были частичные ошибки записи отчетов, динамики или structural parsing, exit_code становится 1; help/version возвращают 0; ошибки аргументов - 2.

Результат шага: процесс завершен, все файлы отчетов либо записаны, либо ошибка явно сообщена.

3. Как понимать “для чего” весь этот маршрут

Проект специально устроен как конвейер доверия. Он не прыгает сразу к “аномалия есть/нет”. Сначала он доказывает, что умеет правильно найти координатную систему PE-файла. Потом доказывает, что может безопасно прочитать официальную exception directory. Потом разбирает каждую unwind-структуру. Только после этого он имеет право считать признаки.

Логика такая: если повреждена карта секций, подозрителен весь адресный контекст. Если повреждена .pdata, подозрительна карта функций. Если повреждена .xdata, подозрительна логика восстановления стека. Если handler лежит в странной секции или один handler доминирует, подозрительна exception-dispatch картина. Если несколько таких вещей встречаются вместе, итоговый вывод усиливается.

Поэтому проект является системой анализа, а не голым парсером. Парсер отвечает только за чтение формата. Здесь же есть проверка инвариантов, вычисление признаков, корреляция признаков, оценка подозрительности, confidence, динамическое evidence и отчеты.

4. Что именно проект НЕ делает

Он не эмулирует весь процессор и не доказывает математически, что бинарник вредоносный. Он не заменяет полноценный reverse engineering. Он не анализирует PE x86 как основную цель. Он не видит JIT-таблицы, которые появились только в runtime, если не включен Windows dynamic layer и если путь исполнения не привел к регистрации этих таблиц. Его задача уже и точнее: найти нарушения и нетипичные паттерны в PE x64 EH/unwind metadata и дать воспроизводимый отчет.

5. Итоговая формула для защиты

Если спрашивают, что проект делает пошагово, ответ такой: “Сначала программа валидирует вход и режим запуска. Затем читает PE-файл в память. Потом проверяет PE32+ AMD64-заголовки и строит карту секций. После этого находит Exception Directory, читает массив RUNTIME_FUNCTION из .pdata, по каждой записи переходит к UNWIND_INFO в .xdata, разбирает unwind-коды, handler и chain. Потом сравнивает полученную структуру с инвариантами Microsoft x64 unwind, считает признаки повреждения и нетипичного размещения, объединяет признаки в score/confidence/verdict. Если включена динамика на Windows, дополнительно запускает target под debugger и собирает runtime-события по исключениям и function tables. В конце пишет JSON, HTML, CSV и Ghidra CSV”.

Самый короткий вывод: проект автоматизирует не действия человека, а технический конвейер проверки EH/unwind-метаданных: от байтов PE-файла до объяснимого verdict и отчетов.

4. Реальная польза проекта для реверс-инженера

Этот раздел объясняет не будущий идеальный инструмент, а текущую практическую ценность проекта в его существующем виде. Важно формулировать честно: проект не заменяет реверс-инженера и не делает полную семантическую реконструкцию исполнения. Его ценность в другом: он автоматически собирает, проверяет и структурирует тот слой EH/Unwind evidence, который специалист обычно вынужден извлекать вручную из PE headers, .pdata, .xdata, секций, handler-адресов и динамических событий.

Коротко: проект дает реверсеру не окончательную истину, а подготовленную карту доказательств. Он показывает, куда смотреть, какие структуры связаны, какие поля нарушают инварианты, какие адреса подозрительны, какие сигналы усиливают друг друга, а какие требуют осторожной ручной проверки.

4.1. Главная ценность: быстро построить карту EH/Unwind metadata

Windows x64 exception/unwind metadata распределена по нескольким слоям: PE headers задают адресное пространство образа, sections задают mapping RVA -> file offset и права памяти, Exception Directory указывает на .pdata, .pdata содержит RUNTIME_FUNCTION, а каждая RUNTIME_FUNCTION ведет к UNWIND_INFO в .xdata или другой metadata-секции. Вручную это можно разобрать, но это медленно и легко ошибиться в RVA, alignment, CountOfCodes, padding или tail layout.

Ручная работа реверсера	Что делает проект	Практический выигрыш
Найти Exception Directory и понять, где .pdata.	Читает PE headers, DataDirectory, exception_rva и exception_size.	Специалист сразу видит, есть ли таблица функций и где она расположена.
Проверить секции и права.	Строит список sections: name, VA, VS, raw offset, raw size, R/W/X.	Сразу видно, где код, metadata, writable/executable области и странное размещение.
Разобрать RUNTIME_FUNCTION вручную.	Читает begin_rva, end_rva, unwind_rva для каждой функции.	Получается карта функций Windows x64 без ручного чтения .pdata.
Перейти из .pdata в .xdata.	Мапит UnwindInfoAddress, разбирает UNWIND_INFO, сохраняет error per function.	Не нужно руками высчитывать file offset для каждой записи.
Понять handler и chain.	Показывает flags, has_handler, handler_rva, handler_section, has_chain, chain.	Можно быстро выделить функции, связанные с exception handling или chained metadata.

4.2. Ценность для первичного triage

Triage - это первичная сортировка и отбор того, что стоит смотреть глубже. Реверсер редко начинает с полного ручного анализа всех функций. Сначала он хочет понять, где необычное, где нарушены инварианты, где много handler-ов, где metadata лежит в странных секциях, где один handler доминирует над большим числом функций. Проект автоматизирует именно этот первый слой отбора.

- Быстро показывает, есть ли грубые структурные повреждения PE: section OOB, overlaps, misalignment, exception directory OOB.
- Показывает проблемы таблицы RUNTIME_FUNCTION: invalid ranges, выходы за SizeOfImage, пересечения, неотсортированность, duplicate BeginAddress.
- Показывает проблемы UNWIND_INFO: bad opcode, bad operand, truncation, code offset за пределами prolog, chain loop/depth/invalid, alignment issues.
- Выделяет подозрительное размещение metadata: UNWIND_INFO в executable, writable или необычной секции.
- Выделяет подозрительное размещение handler-а: non-executable, unknown или writable секция.
- Считает reuse/dominance: насколько активно функции переиспользуют один UnwindInfoAddress или один handler target.
- Дает score, severity, confidence и evidence families, чтобы не читать все подряд с одинаковым приоритетом.

4.3. Ценность как средство против человеческих ошибок

Большая часть ручного анализа PE/EH/Unwind опасна не из-за невозможности, а из-за мелких ошибок. Реверсер может перепутать VA и RVA, забыть про padding после нечетного CountOfCodes, принять operand slot за новую UWOP, прочитать handler RVA не с того места, неправильно сопоставить RVA с секцией или пропустить chain. Проект снижает именно этот риск.

Типичная ошибка	Как проект снижает риск
Перепутать VA/RVA/file offset.	Работает с PE image model и делает RVA mapping через секции, показывает representative_file_offset там, где он есть.
Считать один UWOP = один slot.	Учитывает multi-slot операции: ALLOC_LARGE, SAVE_NONVOL, SAVE_XMM128 и far-варианты.
Забыть padding перед хвостом UNWIND_INFO.	Считает выровненный массив slots_even и только после него читает handler/chained tail.
Пропустить обрыв .xdata.	Фиксирует truncation и parse_error per function.
Прочитать CHAININFO как handler или наоборот.	Разделяет flags EHANDLER/UHANDLER/CHAININFO и проверяет недопустимые комбинации.
Не заметить handler в неисполняемой секции.	Показывает handler_section и handler_nonexec_section finding.
Не заметить metadata в writable секции.	Показывает unwind_writable_section и metadata_placement findings.

4.4. Ценность для работы в Ghidra

Даже если опытный реверсер предпочитает Ghidra, проект остается полезным. Он не заменяет Ghidra, а подготавливает для нее карту функций и комментарии. Это сокращает путь от сырого PE к точкам ручной проверки.

1. Проект экспортирует Ghidra CSV с begin_rva, end_rva, name и unwind-комментарием.
2. Ghidra importer может создать или дополнить функции по ranges из RUNTIME_FUNCTION.
3. Комментарии дают сводку: version, flags, prolog, stack, frame, saved registers, handler_rva, chain, anomalies.
4. Реверсеру проще открыть конкретный representative RVA из finding и сразу понять, с какой function/unwind записью он связан.
5. Если finding указывает на handler, можно быстро перейти к handler target и проверить, является ли он реальным кодом, thunk, import-like переходом или подозрительной областью.

Практический смысл: проект помогает превратить Ghidra-сессию из “ищу, где здесь .pdata и что она значит” в “проверяю заранее отмеченные адреса и гипотезы”.

4.5. Ценность findings: не просто адрес, а причина

Проект полезен не только потому, что показывает адреса. Он дает structured findings. Это важнее обычного дампа: у finding есть код, семейство, severity, affected_count, representative RVA, section, file offset, related function, rule, observed, expected и impact. Такая структура помогает реверсеру понимать, почему место попало в отчет.

Компонент finding	Польза для реверсера
code	Можно фильтровать и сравнивать одинаковые признаки между файлами.
family	Понятно, к какому слою относится проблема: table_integrity, unwind_integrity, metadata_placement, reuse, handler dominance.
severity	Сразу видно, что проверять первым.
affected_count	Отличает единичный дефект от массового паттерна.
representative_rva	Дает точку входа для Ghidra/hex viewer/debugger.
representative_file_offset	Позволяет открыть место в файле, если RVA мапится.

Компонент finding	Польза для реверсера
related_function_*	Связывает проблему с конкретной функцией и ее unwind_rva.
rule/observed/expected	Показывает не только факт, но и критерий: что должно быть и что реально найдено.
impact	Объясняет, почему это влияет на stack walking, exception dispatch, trust к metadata или подозрительность layout.

4.6. Ценность динамического слоя

Динамический слой полезен не потому, что полностью объясняет весь маршрут исполнения, а потому что добавляет runtime evidence поверх статической карты. Static показывает, что файл обещает. Dynamic показывает, что произошло в конкретном запуске под отладчиком. Между этими слоями бывает расхождение, и проект помогает его увидеть.

Что видит dynamic	Практическая польза
first/second chance exceptions	Показывает, возникали ли исключения и на какой фазе они наблюдались.
unwind handler hits	Показывает, дошло ли исполнение до handler-ов, связанных со статической UNWIND_INFO.
VEH/VCH/UEF registrations and hits	Помогает отличить frame-based EH от vectored/global exception маршрутов.
RtlAddFunctionTable / RtlInstallFunctionTableCallback	Фиксирует runtime-регистрации function tables, важные для JIT/runtime-generated code.
RIP trace after handler hit	Дает ограниченный след адресов после попадания в handler, чтобы реверсер мог проверить дальнейший маршрут.
instrumentation counters and dropped events	Показывает, насколько полным было наблюдение и не потеряны ли события.

Важная честная граница: текущая динамика не строит полную narrative route “RIP -> function -> handler -> next RIP -> interpretation”. Но она дает сырье для такой реконструкции: события, адреса, counters, trace и признаки корреляции со статикой.

4.7. Ценность для понимания “что файл обещает”

Статический отчет показывает договор, который PE-файл предлагает Windows: какие диапазоны считаются функциями, где лежит unwind metadata, есть ли handler, где он находится, есть ли chain, как устроен stack frame по metadata. Это очень важный слой, потому что Windows stack walking и exception dispatch доверяют этим таблицам.

Для реверсера это означает: если metadata выглядит поврежденной или странно размещенной, это может влиять на восстановление стека, на обработку исключений, на отладку, на crash analysis и на понимание control flow. Даже если маршрут никогда не исполнялся в конкретном запуске, сама карта файла может показать следы упаковки, ручной модификации, обфускации или необычного runtime.

4.8. Ценность для сравнения файлов и версий

JSON-отчет удобен не только для одного просмотра. Его можно сохранять в базу данных и сравнивать между версиями бинарника, между clean/mutated samples, между benign corpus и suspicious corpus.

- Сравнивать SHA-256 и report_metadata, чтобы не путать файлы.
- Сравнивать количество RUNTIME_FUNCTION entries и размеры функций.
- Сравнивать распределение unwind_rva и handler_rva.
- Сравнивать score, confidence, findings by code и affected_count.
- Сравнивать появление writable/executable metadata placement после упаковки или патча.
- Сравнивать static findings с dynamic events по разным сценариям запуска.

Это делает проект полезным не только как разовый CLI-инструмент, но и как ядро для backend/DB/frontend: ядро анализирует, backend сохраняет, frontend показывает историю и сравнения.

4.9. Польза для новичка и для опытного реверсера разная

Пользователь	Какая польза
Новичок	Получает структуру анализа: какие поля важны, как связаны PE sections, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, handler, chain. Видит manual steps и учится не прыгать сразу к выводу.
Студент на защите	Может объяснить, что проект автоматизирует ручной structural workflow, показывает evidence, ограничения и дальнейшее развитие.
Опытный реверсер	Быстро получает карту подозрительных мест, адреса, file offsets, Ghidra import, findings and dynamic counters. Он может не нуждаться в объяснении терминов, но ценит скорость и стандартизацию.
Разработчик backend/frontend	Получает стабильный JSON-контракт, который можно сохранять, фильтровать, сравнивать и визуализировать.
Заказчик/аналитик	Получает HTML/PDF-пояснения, score, findings and manual validation plan вместо набора несвязанных hex-адресов.

4.10. Что проект не дает и почему это важно не скрывать

Практическая ценность проекта не должна защищаться через преувеличение. Наоборот, сильнее звучит честная граница: проект хорош как структурный анализатор и triage-система, но не как полный semantic reverse engineering assistant.

Не дает сейчас	Что это означает для реверсера
Не сравнивает UWOP с инструкциями пролога.	Если нужно доказать соответствие push/sub/mov в .text, специалист проверяет это в Ghidra вручную или это будет будущий prolog-vs-unwind validator.
Не симулирует RtlVirtualUnwind по реальному стеку.	Не показывает восстановление caller RIP/registers из stack memory. Для этого нужен отдельный контекст и simulator.
Не реконструирует C SEH scope table.	Видит handler RVA, но не строит цепочку filter -> handler target для __C_specific_handler.
Не решает за человека benign/malicious окончательно.	Score и findings - это triage/evidence, а не окончательный экспертный приговор.
Не покрывает весь malware analysis.	Область проекта специализирована: PE x64 EH/Unwind metadata.

4.11. Почему ценность все равно существенная

Утверждать, что ценность проекта мала, было бы неверно. Небольшая ценность была бы у инструмента, который просто пишет “SEH detected” или печатает пару flags без адресов и контекста. Здесь ценность выше: проект строит карту, проверяет инварианты, связывает объекты, формирует findings, дает severity, confidence, impact, manual steps и экспорт в форматы, пригодные для дальнейшей работы.

Но также неверно утверждать, что проект уже полностью объясняет поведение исключений. Поэтому самая честная оценка: текущая ценность для реверс-инженера хорошая на этапе первичного структурного анализа, подготовки evidence, отбора подозрительных мест и стандартизации отчета; максимальной она станет после добавления prolog-vs-unwind validation, handler data parser and narrative static/dynamic route reconstruction.

4.12. Формулировка для защиты

> “Практическая ценность проекта для реверс-инженера состоит в автоматизации трудоемкого и ошибкоопасного слоя анализа PE x64 EH/Unwind metadata. Проект строит адресную карту PE, читает .pdata и .xdata, сопоставляет функции с UNWIND_INFO, выделяет handler-ы и chain, проверяет структурные инварианты, выявляет подозрительное размещение и reuse, формирует findings с адресами, секциями, expected/observed/impact, а также дает JSON/HTML/CSV/Ghidra CSV для дальнейшей работы.”

> “Проект не заменяет реверс-инженера. Он готовит evidence и показывает, где нужно думать. Финальная интерпретация остается за специалистом, потому что только человек может учесть

> компилятор, runtime, packer, JIT-сценарий, custom handler и контекст исполнения. Поэтому проект является не автоматическим реверсером, а инструментом автоматизированного структурного анализа и triage.”

4.13. Короткий ответ на вопрос “зачем он нужен опытному реверсеру?”

- Чтобы быстрее получить карту EH/Unwind metadata без ручного чтения каждой .pdata/.xdata записи.
- Чтобы не ошибиться в RVA/file offset, CountOfCodes, padding, multi-slot UWOP, handler tail и chain.
- Чтобы сразу увидеть функции с parse failures, anomalies, handler-ами, chain и suspicious placement.
- Чтобы получить Ghidra CSV и перейти к проверке конкретных адресов, а не искать их вручную.
- Чтобы иметь JSON/HTML evidence report, который можно сохранить, сравнить, показать коллеге или использовать во frontend/DB.
- Чтобы дополнить static analysis динамическими событиями: exceptions, handler hits, VEH/VCH/UEF, runtime function table API calls.
- Чтобы стандартизировать первичный анализ и не превращать каждую проверку в разовый ручной ритуал.

4.14. Итог

В текущем виде проект полезен реверс-инженеру как подготовительный и аналитический слой. Его задача - быстро превратить бинарник в структурированную карту EH/Unwind evidence. Он не рассказывает всю историю исполнения, но дает адреса, связи, нарушения, приоритеты и форматированный отчет. Это именно тот слой, который обычно отнимает много времени перед глубоким ручным анализом в Ghidra/debugger.

5. Медленная последовательность вопросов по странной .pdata

Когда реверс-инженер видит странную .pdata, ему нельзя начинать с общего вывода “тут SEH” или “тут все плохо”. Правильный путь - идти по последовательности вопросов. Такая последовательность удерживает уровни адресации: файл, image, RVA, file offset, секции, .pdata, .xdata, handler, dynamic evidence. Проект полезен именно потому, что большую часть этой последовательности он автоматизирует и превращает в отчетные поля, счетчики и findings.

Главная идея: проект не отменяет размышление специалиста, но дисциплинирует его. Он заставляет анализ идти от координатной системы PE к таблице функций, от таблицы функций к UNWIND_INFO, от UNWIND_INFO к handler/chain, от handler/reuse к динамическим подтверждениям.

5.1. Чеклист: какие вопросы закрывает проект

Вопрос реверсера	Делает?	Что именно дает проект
Где находится Exception Directory? Его RVA и Size корректны?	Да	Показывает exception_directory.rva/size, проверяет exception_directory_oob. Это первая точка входа в .pdata.
Размер Exception Directory/.pdata кратен 12 байтам?	Да	RUNTIME_FUNCTION имеет размер 12 байт. Если directory structurally unreadable или malformed, проект фиксирует проблему чтения exception directory/partial structural report.
Какой ImageBase у модуля? Не перепутаны ли VA, RVA и file offset?	Да, частично	Показывает ImageBase и работает в модели RVA. Для findings выводит representative_rva и representative_file_offset. Но он не обучает интерактивно VA/RVA, а сам использует правильную адресную модель.
Какие секции покрывают .pdata и .xdata? Какие у них read/write/execute права?	Да	Показывает sections[], права R/W/X, unwind_section, handler_section, metadata placement findings.
Все ли RUNTIME_FUNCTION имеют BeginAddress < EndAddress?	Да	Проверяет invalid ranges через rf_invalid_ranges и finding invalid_ranges.
Отсортирована ли таблица по BeginAddress? Есть ли duplicate begins?	Да	Проверяет rf_unsorted и rf_duplicate_begins. Это важно для однозначного поиска функции по ControlPc.
Есть ли overlaps?	Да	Проверяет rf_overlaps. Показывает affected_count и representative_rva.
Если overlaps есть, какие записи конфликтуют и разные ли у них UNWIND_INFO?	Частично	Факт overlaps фиксируется, но богатой таблицы “entry A конфликтует с entry B, unwind_rva такие-то” сейчас нет. Это развитие отчета.
Есть ли gaps в executable code? Похожи ли они на padding, thunks или настоящий код?	Нет	Текущий проект не дизассемблирует .text и не классифицирует gaps. Для этого нужен code/prolog/thunk analyzer.
Куда ведет каждый UnwindInfoAddress? Парсится ли UNWIND_INFO?	Да	Показывает unwind_rva, unwind_section, unwind_ok, unwind_error, unwind.version/flags/prolog/count/stack/anomalies.
Есть ли EHANDLER, UHANDLER, CHAININFO?	Да	Разбирает flags, показывает has_handler, handler_rva, has_chain, chain.
Где лежит handler target?	Да	Показывает handler_rva и handler_section, ловит handler_nonexec_section и handler_writable_section.
Есть ли массовый reuse или handler dominance?	Да	Считает unwind_reuse_ratio, unwind_unique_ratio, handler_count, handler_reuse_ratio; формирует findings по reuse/dominance.
Это стандартный language handler или нестандартный dispatcher?	Нет/частично	Проект может заметить dominance и suspicious placement, но не распознает надежно __C_specific_handler/C++ EH/custom dispatcher. Это остается реверсеру или будущему handler_kind module.
Что показывает динамика: exceptions, handler hits, dynamic function tables, RIP trace?	Да	В --dynamic на Windows показывает exceptions, first/second chance, unwind_handler_hits, VEH/VCH/UEF, RtlAddFunctionTable, RtlInstallFunctionTableCallback, trace, dropped_events.

5.2. Что это значит простыми словами

Из этого чеклиста видно: проект закрывает почти весь базовый структурный маршрут анализа .pdata/.xdata. Он хорошо отвечает на вопросы “где лежит”, “мапится ли”, “разбирается ли”, “какие flags”, “есть ли handler”, “есть ли chain”, “есть ли reuse”, “есть ли подозрительное размещение”, “есть ли динамические события”. Это уже существенная часть ручной работы реверсера.

Но есть семантические вопросы, которые текущая версия не закрывает полностью: конфликтующие пары overlap не раскрываются как подробная таблица, gaps в executable code не классифицируются, language handler kind не определяется надежно, dispatcher logic не реконструируется. Это не ломает текущую ценность проекта, но показывает честные направления развития.

5.3. Как реверсер использует этот checklist на практике

1. Сначала он смотрит Exception Directory и секции, чтобы убедиться, что адресная карта файла вообще надежна.
2. Потом смотрит .pdata: диапазоны функций, сортировка, invalid ranges, overlaps, duplicate begins.
3. Потом переходит к .xdata: парсится ли UNWIND_INFO, какие flags, prolog_size, CountOfCodes, stack effect, anomalies.
4. Потом выделяет функции с handler-ами и chain, смотрит handler_rva, handler_section и права секции.
5. Потом смотрит агрегаты: reuse ratio, handler dominance, evidence families, severity и score.
6. Потом, если есть Windows dynamic run, он сравнивает статические ожидания с runtime evidence: exceptions, handler hits, VEH/VCH/UEF, dynamic function table registrations, trace.
7. И только после этого делает экспертную интерпретацию: benign compiler/runtime layout, packing/protection, corrupted metadata, exception-based route или ручное вмешательство.

5.4. Какие пункты особенно сильны для защиты

Сильный пункт	Почему он важен на защите
Проект закрывает уровни адресации	Комиссия может спросить про VA/RVA/file offset. Проект показывает ImageBase, sections, RVA mapping and file offsets for findings.
Проект проверяет .pdata как таблицу структур, а не просто печатает ее	Есть invalid ranges, OOB, sorting, overlaps, duplicate begins. Это больше, чем parser.
Проект разбирает .xdata с учетом деталей формата	Есть CountOfCodes, multi-slot UWOP, padding, tail, handler/chain, parse errors.
Проект показывает placement	Права секций R/W/X важны: metadata в writable/executable области и handler в non-executable области - реальные поводы для review.
Проект считает reuse/dominance	Массовое переиспользование unwind/handler может указывать на dispatcher-style схему или защитную логику.
Проект добавляет dynamic evidence	Он не остается только static parser: может увидеть exceptions, handler hits and runtime function table API calls.

5.5. Три пункта, которые надо честно назвать ограничениями

Ограничение	Как правильно объяснить
Gaps в executable code	Текущий проект не анализирует промежутки между RUNTIME_FUNCTION как padding/thunk/real code, потому что для этого нужен анализ .text и дизассемблирование.
Overlap details	Проект фиксирует overlaps как finding, но не показывает полную матрицу конфликтующих пар. Это можно добавить без переписывания ядра как улучшение отчета/evidence.
Language handler identity	Проект видит handler RVA and placement, но не утверждает, что это __C_specific_handler или custom dispatcher без надежного распознавания. Это правильно, потому что ошибочное распознавание даст ложную уверенность.

5.6. Как это сказать на защите

> “Проект автоматизирует основную дисциплинированную последовательность вопросов, которую реверс-инженер задает при анализе странной .pdata: где находится Exception Directory, корректны ли размеры и RVA, какие секции покрывают metadata, валидны ли RUNTIME_FUNCTION ranges, отсортирована ли таблица, есть ли overlaps/duplicates, куда ведет UnwindInfoAddress, парсится ли UNWIND_INFO, есть ли EHANDLER/UHANDLER/CHAININFO, где расположен handler, есть ли reuse/dominance и что подтверждает динамика.”

> “При этом проект честно оставляет специалисту семантические вопросы: gaps в .text, классификацию thunk/padding/real code, точное распознавание language handler-а и dispatcher

> logic. Поэтому проект является не заменой реверсера, а инструментом, который стандартизирует и ускоряет структурный этап анализа.”

5.7. Итог по этому checklist

Текущий проект делает большую часть checklist-а, связанного с PE layout, .pdata, .xdata, handler placement, reuse and dynamic counters. Его слабые места находятся не в базовой структурной проверке, а в более глубоком semantic layer: анализ .text gaps, подробная overlap evidence matrix, handler identity and language-specific handler data. Именно поэтому проект уже полезен как структурный анализатор, но имеет понятную дорожную карту развития в сторону более умного reverse engineering assistant.